

12 Логик программчлалын парадигм

12.1 Deduction as Computation

Р.Ковальскийн алдартай тунхаглал нь программчлалын үйл явцын үндэс болсон ойлголтуудыг маш зөв тусгасан: Алгоритм = Логик + Хяналт. Алгоритмын тодорхойлолт болон түүний программчлалын хэл дээрх томъёолол нь энэхүү "тэгшитгэл" ёсоор хоёр хэсэгт хуваагдана. Нэг талаас, шийдлийн логикийг зааж өгсөн – өөрөөр хэлбэл "юу" хийсэн байх ёстойг тодорхойлсон. Нөгөө талаас, хяналттай холбоотой асуудлуудыг зааж өгснөөр хүссэн шийдлийг “хэрхэн яаж” олох нь тодорхойлогдсон. Уламжлалт императив хэл хэрэглэж буй програмист нь эдгээр бүрдэл хэсгүүдийг хоёуланг нь харгалзан үзэх ёстой. Нөгөө талаас логик програмчлал нь энэ хоёр талыг тусгаарлах санаагаар анхнаасаа тодорхойлогдсон. Программист зарчмын хувьд зөвхөн логик тодорхойлолт өгөхийг шаарддаг. Хяналттай холбоотой бүх зүйлийг хийсвэр машинд хариуцуулдаг. Тусгайлсан дедукц дүрэм (нарийвчлал) дээр үндэслэсэн тооцооллын механизмыг ашиглан интерпретатор нь "логик"-оор тодорхойлсон боломжит шийдлүүдийн орон зайг хайх байдлаар эцсийн үр дүнд хүрэхэд шаардлагатай үйлдлүүдийн дарааллыг тодорхойлдог.

Тооцооллыг логик дедукц гэж үзэх үндэс нь 1930-аад оны К.Годель, Ж.Хербранд нарын бүтээлээс эхтэй. Ялангуяа Хербранд логик програмчлалын хэлний үндсэн тооцооллын механизм болох нэгтгэл (unification) процессын талаарх зарим санааг бүрэн бүс ч гэсэн таамаглаж байсан.

Энэ процессын формал тодорхойлолт (А.Робинсоны улмаас) 1960-аад он хүртэл гарч ирсэнгүй. Улмаар арван жилийн дараа (1970-аад оны эхээр) энэхүү тохиромжтой формал автомат дедукц нь тооцооллын механизм болгон интерпретац хийж болохыг ойлгов. Prolog (нэр нь LOGic хэл дээрх PROgramming гэсэн үгийн товчлол юм; түүхийн талаарх дэлгэрэнгүй мэдээллийг 16.4-с үзнэ үү) бүтээгдсэнээр энэхүү парадигмын анхны програмчлалын хэлүүд бий болсон.

Өнөөдөр Prolog-ийн олон хувилбар байдаг бөгөөд энэ парадигмд өөр хэд хэдэн хэл байдаг (хэрэглээний хувьд тусгай сонирхлын шаардлагуудыг багтаасан хэлүүд). Эдгээр бүх хэлүүд (Прологыг оруулаад) үр ашгийн үүднээс хяналтыг ил зааж өгөх бүтцийг агуулсан байдаг. Эдгээр бүтцүүд нь шууд логик интерпретацгүй тул хэлний семантикийг илүү төвөгтэй болгож, логик парадигмын цэвэр декларатив шинж чанарыг бууруулдаг. Гэсэн хэдий ч бид тэдгээрийг логик програмчлалын хэл гэж нэрлэдэг хэвээр байгаа бөгөөд тэдгээр нь програмистаас шийдэх ёстой асуудлынхаа тодорхойлолтыг томъёолох (эсвэл зарлах)-аас өөр зүйл хийхийг шаарддаггүй. Зарим тохиолдолд, үр дүнд нь бий болсон программүүд нь товч, энгийн, ойлгомжтой байдлаараа үнэхээр гайхалтай бөгөөд үүнийг бид дараагийн хэсэгт үзэх болно.

Нэр томъёоны тайлбар

Бид онолын суурь формализм болох логик програмчлалыг олон янзаар хэрэгждэг Пролог хэлнээс ялгах ёстой. Бидний доор танилцуулж буй ойлголтууд, хэрэв өөрөөр заагаагүй бол хоёуланд нь хүчинтэй байна. Чухал ялгааг ил зааж өгөх болно. Бидний оруулсан програмчлалын жишээнүүд (ихэвчлэн this фонтоор) нь хэлний зарим хэрэгжилт дээр ажиллуулж болох Пролог кодууд юм. Нөгөө талаас онолын ойлголтууд нь Прологийн хувьд хүчинтэй байсан ч математикийн ойлголтуудыг ашигладаг бөгөөд *налуу үсгээр* бичсэн байдаг. Түүнчлэн, бид үргэлж том үсгээр эхэлсэн хувьсагчдыг бичдэг Прологийн конвенцийг баримтлах болно. Өмнө дурьдсанчлан логик програмчлалын

² Хэрэв Ls нь (хоосон байж магадгүй) жагсаалтыг Li болон өөр жагсаалттай (өөрөө хоосон байж магадгүй) холбосноор Li нь Ls -ийн дэд жагсаалт гэдгийг санаарай.

гэж зааж өгч болно, энд X ба Y нь хувьсагч. Нэргүй хувьсагчийг дахин ашигласнаар бид дараахь зүйлийг бичиж болно.

Бидэнд хоёртын предикатын $\text{sublist}(X,Y)$ байгаа гэж үзвэл эхний аргумент (X) нь хоёр дахь (Y)³ -ын дэд жагсаалт гэсэн утгатай тул бидний шаардлагыг дараах байдлаар илэрхийлж болно:

тоо 2 руу шилжиж, үүнтэй төстэй байдлаар тайлбарлавал Ls жагсаалтад мөн дэд жагсаалтыг агуулсан байх ёстой:

тиймээс дараахь зүйл бас үнэн байх ёстой.

Энэ үндэслэлийг 9 хүртэлх бусад тоонуудын хувьд давтан хэрэглэхэд бидний гаргахыг хүсч буй sol програмыг дараах байдлаар тодорхойлж болно.

Энд урьдын адил :- тэмдгийн баруун талд байгаа хэсэг нь зүүн гар талын тодорхойлолт юм. Энэ програм нь бидний асуудлын шийдлийг олохын тулд (1-р мөрөнд sol(Ls)) 2-11-р мөрөнд тайлбарласан бүх шинж чанаруудыг хангасан байх ёстой гэж хэлж байна. Ялгаатай предикатуудыг тусгаарладаг таслалыг "ба" гэж тайлбарлах нь зүйтэй бөгөөд энэ нь логик утгаараа конъюнкц юм. Бид асуудлын томъёоллыг формалаар давтсан тодорхойлолтыг өгсөн бөгөөд үүнийг цэвэр логик томъёоллоор тайлбарлаж болохыг доор үзэх болно.

³ Харьцааны sublist тодорхойлолтыг 12.4.5-д өгсөн болно.

Өөрөөр хэлбэл, бидний тодорхойлолтыг процедурын зарчмын дагуу дараах байдлаар уншиж болно. 1-р мөрөнд `sol` гэж нэрлэгддэг процедурын зарлалтыг агуулсан; энэ нь `Ls` нэг формал параметртэй. Энэ процедурын үндсэн хэсгийг 2-оос 11-р мөрөнд тодорхойлсон бөгөөд бид арван процедурын дуудалттай. 2-р мөрөнд `list_of_27` процедурыг бодит `Ls` параметрээр дуудна. Түүний тодорхойлолтын дагуу `list_of_27` нь өөрийн бодит параметрийг нэргүй 27 хувьсагчийн жагсаалт болгон үүсгэнэ⁴. Ингэснээр нь 3-р мөрөнд бид дараах дуудалттай байна:

Энэ дуудалт нь Ls хувьсагчийг үүсгэснээр эхний параметр болж харагдах жагсаалт нь хоёр дахь параметртэй (Ls) холбогдсон жагсаалтын дэд жагсаалт байхаар асуудлыг зохицуулдаг гэж бид үзэж байна. 11-р мөр хүртэлх бусад дуудлагын хувьд мөн адил. Параметр дамжуулалт нь нэрээр дууддагтай төстэй байдлаар явагддаг ба Prolog-д өөр өөр процедурын дуудлагуудын текстэнд гарч ирэх дараалал нь үнэлгээний дарааллыг заадаг. (цэвэр логик программуудын хувьд нөгөө талаас ямар ч дарааллыг заагаагүй).

Нэг процедурын дараалсан дүүдлалт нь асуудлын бусад шийдлийг олж авах боломжийг олгодог.

12.2 Syntax

Энд логик нь 1-р эрэмбэ (first-order) логик байгаа бөгөөд үүнийг мөн *предикат тооцоолол* (*predicate calculus*) гэж нэрлэдэг. Сэдвийн тогтмол D домайны элементийн (эсвэл илүү хуучин нэр томьёогоор "урьдчилан таамаглах") шинж чанарыг симбол ашиглан тэмдэглэнэ. Өндөр эрэмбийн логикууд (2,3...-р эрэмбэ) нь D-ийн (2-р эрэмбэ) элементүүдээс гадна D-ийн (3-р эрэмбэ) олонлог ба функцууд, эдгээр функцуудын багц гэх мэт аргументууд нь илүү төвөгтэй объект болох предикатуудыг зөвшөөрдөг.

⁴ Нэр томьёо нь анхнаасаа ойлгомжтой байсан ч: хувьсагчийг үүсгэснээр бид үүнд синтаксийн объектыг орлуулахыг хэлнэ. Бид 12.3-т тодорхой тодорхойлолт өгөх болно.

12.2.1 The Language of First-Order Logic

Нэгдүгээр эрэмбийн логик хэл нь гурван бүрэлдэхүүн хэсгээс бүрдэнэ.

1. Цагаан толгой.
2. Энэхүү цагаан толгойн үсгээр тодорхойлсон нөхцөл.
3. Энэ цагаан толгойн дээр тодорхойлогдсон зөв хэлбэртэй (Well-formed) томъёо (formulae).

Цагаан толгой

Цагаан толгой⁵ нь логик (бүх нэгдүгээр эрэмбийн хэлэнд нийтлэг) тэмдгийн олонлог болон логик бус (эсвэл нэмэлт логик) тэмдгийн олонлог гэсэн хоёр салангид дэд олонлогт хуваагдсан тэмдгийн олонлог юм (сонирхлын домайнд тусгайлан зориулагдсан). Жишээлбэл, бүх нэгдүгээр эрэмбийн хэл нь конъюнкцийг тэмдэглэхийн тулд (логик) тэмдэг ашиглана. Хэрэв бид олонлог дээр эрэмбэлэлт хийх гэж байгаа бол логик бус тэмдгийн нэг болох $<$ тэмдэгтэй байх магадлалтай.

Логик тэмдгийн олонлог нь дараахь элементүүдийг агуулна.

- Логик холбогч $\wedge$ (конъюнкц), $\vee$ (дизъюнкц), $\neg$ (үгүйсгэл), $\rightarrow$ (импликац) ба $\leftrightarrow$ (давхар импликац).
- Хэллэгийн тогтмолууд үнэн ба худал.
- $\exists$ (оршин байх) ба $\forall$ (дурын) кванторууд.
- “(” ба “)” хаалт, таслал “,” гэх мэт зарим тусгаарлах тэмдэг.
- $X, Y, Z, \dots$ гэж бичигдсэн хувьсагчийн хязгааргүй V олонлог (тоолох боломжтой)

Логик бус (эсвэл өргөтгөсөн логик) тэмдэг нь *предикат бүхий гарын үсгээр* (Σ, Π) тодорхойлогддог. Энэ нь эхний элемент болох Σ нь *функцийн гарын үсэг*, өөрөөр хэлбэл функцийн тэмдгийн олонлог бөгөөд тус бүр нь өөрийн аргументын тоотой (arity)⁶ хамт байдаг.

Хосын хоёрдахь элемент Π нь *предикатын гарын үсэг*, өөрсдийн аргументын тооны хамт байх предикатын тэмдгийн олонлог юм. Аргументын тоо 0 функцийг *тогтмол* гэж нэрлэдэг ба $a, b, c, \dots$ үсгээр тэмдэглэнэ. Аргументын тоо эерэг функцын тэмдгийг ердийнхөөрөө $f, g, h, \dots$ гэж тэмдэглэдэг бол предикат тэмдгийг $p, q, r, \dots$ гэж тэмдэглэдэг. Σ ба Π олонлогууд нь хоосон огтлолцолтой, мөн логик тэмдгүүдээс салангид байна гэж үзнэ. Функц ба предикатын тэмдгийн ялгаа нь эхнийх нь функц гэж тайлбарлагдах ёстой бол сүүлийнх нь харьцаа гэж тайлбарлагдах ёстой. Формулагийн тухай хэлэлцэх үед энэ ялгаа илүү тодорхой болно.

Нөхцөл

Математик логик болон компьютерийн шинжлэх ухааны үндэс суурь болох нөхцлийн тухай ойлголтыг олон нөхцөл байдалд далд хэлбэрээр ашигладаг. Жишээлбэл, арифметик илэрхийлэл нь операндуудад (арифметик) операторуудыг хэрэглэснээр олж авсан нэр томъёо юм. Тэмдэгт мөр, хоёртын мод, жагсаалт гэх мэт бусад төрлийн бүтцийг тохирох байгуулагчийг ашиглан олж авсан нэр томъёо гэж үзэхэд хялбар байдаг. Семантикийн хувьд нэр томъёо нь сэдвийн хүрээний элементийг илэрхийлдэг Д.

⁵ Бидний энд авч үзэж буй бүх олонлогууд нь төгсгөлөг, тоолох боломжтой юм.

⁶ Дээр дурдсанчлан arity нь функц эсвэл харилцааны аргуудын тоог илэрхийлдэг гэдгийг санаарай.

Синтаксийн хувьд хамгийн энгийн тохиолдолд тухайн нэр томьёо нь түүний аргументын тооны шаардлагын дагуу олон хувьсагч, тогтмолуудад функцийн тэмдэг хэрэглэх замаар олж авдаг. Жишээлбэл, хэрэв a ба b тогтмол, X ба Y нь хоёулаа хувьсагч, f ба g нь 2 аргументтай бол $f(a, b)$ болон $g(a, X)$ нь нэр томьёо юм. Аргументын тоог харгалзсан тохиолдолд функцийн аргумент болгон нэр томьёог ашиглахад юу ч саад болохгүй. Бид жишээ нь $g(f(a, b), Y)$ эсвэл $g(f(a, f(X, Y)), X)$ гэх мэтийг бичиж болно.

Хамгийн ерөнхий тохиолдолд бид нөхцөлийг дараах байдлаар тодорхойлж болно.

Тодорхойлолт 12.1 (Нэр томьёо) Σ гарын үсгийн (мөн хувьсагчийн V олонлог) дээрх нэр томьёог индуктив байдлаар⁷ дараах байдлаар тодорхойлно:w

- Хувьсагч (V дэх) нь нэр томьёо юм.
- Хэрэв f (Σ дэх) нь n аргумент функцийн тэмдэг, $t_1, \dots, t_n$ нь нөхцөл бол $f(t_1, \dots, t_n)$ нь нөхцөл юм.

Хоёр дахь хэсэгчилсэн тохиолдлын хувьд тогтмол нь нөхцөл юм. Тодорхойлолтын үсгийн дагуу тогтмолд харгалзах нэр томьёог хаалттай бичнэ: $a()$, $b()$, Уншихад хялбар болгох үүднээс 0 аргументийн функцийн тэмдгийн хувьд хашилтыг орхиж болохыг тогтооцгооё. Хувьсагчгүй нөхцлийг *үндсэн (ground)* нэр томьёо гэнэ. Нөхцлийг ихэвчлэн $s, t, u, \dots$ үсгээр тэмдэглэдэг. Предикатууд нөхцөл байдаггүй анхаарна уу; Тэд томьёогоор (нөхцлийн талбарыг илэрхийлэх) гарч ирдэг.

Formulae

Хэлний зөв формула (эсвэл товчоор формула) нь нөхцлийн шинжийг илэрхийлэх боломжийг бидэнд олгодог. Тиймээс семантикийн хувьд манай домэйны зарим элемент D нь шинжтэй гэж томъёолсон байдаг. Жишээлбэл, хэрэв бидэнд $>$ (D -д хэсэгчилсэн дарааллын харьцаа гэж ердийн байдлаар тайлбарласан) өгөгдлийн үг байвал $>$ ($3, 2$) гэж бичвэл бид " 3 " гэсэн нэр томьёо нь " 2 " гэсэн нэр томьёотой холбоотой утгаас их утгатай (D -ийн элемент) тохирч байгааг илэрхийлнэ. Логик тэмдэглэгээг ашиглан комплекс томьёог үүсгэж болно. Жишээлбэл, $>$ (X, Y) $\wedge$ $>$ (Y, Z) $\rightarrow$ $>$ (X, Z) томьёо нь $>$ -ийн тусах чанар, — хэрэв $>$ (X, Y) нь үнэн, $>$ (Y, Z) нь мөн үнэн бол энэ нь $>$ (X, Z) гэсэн тохиолдол юм.

Формулаг нарийн тодорхойлохыг хүсвэл бид анхны атомик формула (эсвэл атомууд)-ийг түүний аргументын тооны шаардлагатай томьёоны тоонд предикат хэрэглэх замаар бүтээсэн. Жишээлбэл, хэрэв p нь 2 аргументтэй бол дээр дурдсан хоёр нэр томьёог ашиглан бид $p(f(a, b), f(a, X))$ гэж бичиж болно. Логик холболт, кванторыг ашиглан бид атомик томьёоноос комплекс томьёог гаргаж чадна. Ердийнх шигээ бид индуктив тодорхойлолттой (эсвэл үүнтэй эквивалент, чөлөөт хэлзүй - Дасгал 1-ийг үзнэ үү).

Тодорхойлолт 12.2 (Формула) Гарын үсэг дээрх (Σ, Π) томьёотой (зөв) формулаг индуктив байдлаар дараах байдлаар тодорхойлно:

1. Хэрэв $t_1, \dots, t_n$ нь Σ гарын үсэг дээрх томьёо, $p \in \Pi$ нь аргументын тоо n -ийн предикат тэмдэг бол $p(t_1, \dots, t_n)$ нь формула болно.
2. Үнэн ба худал нь формула.
3. Хэрэв F ба G нь формула бол $\neg F$, $(F \wedge G)$, $(F \vee G)$, $(F \rightarrow G)$ ба $(F \leftrightarrow G)$ нь формула болно.

4. Хэрэв F нь формула, X нь хувьсагч бол $\forall X.F$ ба $\exists X.F$ нь формула болно.

12.2.2 Logic Programs

Сүүлийн тодорхойлолтын дагуу бүтээгдсэн томъёо нь маш комплекс бүтэцтэй байж болно. Энэ хүндрэл нь ерөнхийдөө баталгаа олоход хийгддэг оролдлогыг тодорхойлдог. Логик програмчлалд томъёоны тусгай классуудыг тодорхойлсон байдаг бөгөөд энэ нь автоматаар теорем батлах боловсруулалтыг илүү үр дүнтэй хийх боломжийг олгодог. Эдгээр классын нэг нь *resolution* гэдэг гаргалгааны тусгай дүрмээр удирддаг нэгэн *клаус*. Бид *bodum (definite) клаус* гэдэг клаусын ойлголтын хязгаарлалт болон, мөн түүнчлэн *SLD resolution* гэсэн задаргааны хязгаарлагдмал хэлбэрийг сонирхож байна, 12.4.3-ыг үзнэ үү. Эдгээр хоёр ойлголтыг ашиглан баталгааг олох процедур нь маш энгийн төдийгүй баталгаанд шаардлагатай хувьсагчдын утгыг ил тооцоолох боломжийг олгодог. Эдгээр утгыг тооцооллын үр дүн гэж үзэж болох бөгөөд энэ нь логик дедүкц дээр суурилсан тооцооллын сонирхолтой загварыг бий болгож байна. Бид энэ загварыг доороос илүү нарийвчлан үзэх болно, одоо бид синтаксик тал дээр анхаарлаа хандуулах болно.

Тодорхойлолт 12.3 (Логик программ) $H, A_1, \dots, A_n$ атомик томъёо байг. Тодорхой clause (бидний хувьд зүгээр л "clause") нь дараах хэлбэрийн томъёо юм:

$$H : -A_1, \dots, A_n.$$

Хэрэв $n = 0$ бол клаусыг *нэгж* гэж хэлэх буюу факт, бөгөөд “:–” симболыг орхисон (гэхдээ бүх төгсгөлийг биш). Логик программ нь клаусын олонлог байдаг бол цэвэр Пролог програм нь клаусын эрэмбэтэй цуваа юм. Query (эсвэл зорилго) нь $A_1, \dots, A_n$ атомуудын цуваа.

Энэ тодорхойлолтын талаар зарим зүйлийг тодруулцаая. Нэгдүгээрт, бидний цагаан толгойн үсэгт оруулаагүй “:–” тэмдэг нь (үрвүү) импликац ($\leftarrow$) юм.⁸

Клаус эсвэл query дахь таслалыг логик холболт гэж тайлбарлах хэрэгтэй. Тиймээс “ $H : -A_1, \dots, A_n.$ ” гэсэн тэмдэглэгээ нь “ $H \leftarrow A_1 \wedge \dots \wedge A_n.$ ” гэсний товчлол юм. Цэг нь тэмдэглэгээний нэг хэсэг бөгөөд энэ нь боломжит интерпретатор эсвэл компиляторт клаус дуусч байгааг хэлдэг тул чухал гэдгийг анхаарна уу.

“:–” -ний зүүн талд байгаа хэсэг нь клаусын *толгой (head)*; баруун талд нь *бие (body)* байна. Тиймээс факт нь хоосон биетэй клаус юм. Программ нь онолын формализмын хувьд клаусын олонлог юм. Харин Prolog-ийн хувьд программыг дараалал гэж үздэг, учир нь бидний харж байгааг клаусын дараалал харилцан хамааралтай байдаг. Энд бид сүүлийн үеийн олон бичвэрт байдаг хялбаршуулсан нэр томъёог ашигласан (жишээлбэл, [1]). Илүү нарийн нэр томъёо авахыг хүсвэл дараагийн box-ийг үзнэ үү. Тэдний толгойд p предикат тэмдгийг агуулсан клаус олонлогийг p -ийн *тодорхойлолт* гэнэ. Клаусын толгойд биш биед гарч буй хувьсагчдыг локал хувьсагч гэнэ.

Клаус

Нэгдүгээр эрэмбийн логикийг мэддэг уншигч тодорхойлолт 12.3-т өгөгдсөн клаусын тухай ойлголтыг логикт ашигласан нэг тохиолдол гэж хүлээн зөвшөөрсөн байх болно. Клаус нь ерөнхий утгаараа үнэхээр дараах хэлбэрийн томъёо:

⁸ : – -ыг $\leftarrow$ -ийн оронд практик шалтгаанаар ашиглагддаг. Логик хэлийг нэвтрүүлж байх үед ихэнх гар дээр зүүн сум гэхээсээ илүү: - гэж бичих нь илүү хялбар байсан.

$\forall X_1, \dots, X_m (L_1 \vee L_2 \vee \dots \vee L_n)$.

Энд $L_1 \dots L_n$ нь литерал (атом, эсвэл үгүйсгэсэн атом) ба $X_1, \dots, X_n$ нь $L_1 \dots L_n$ -д тохиолддог бүх хувьсагч юм. Тодорхой болгохын тулд бид үгүйсгэсэн атомуудыг бусдаас нь салгаж болох тул дараах клаусыг бичиж болно.

$\forall X_1, \dots, X_m (A_1 \vee A_2 \vee \dots \vee A_m, \neg B_1 \vee \neg B_2 \vee \dots \vee \neg B_k)$.

Алдартай логик эквивалентуудыг ашиглан бид энэ томъёог дараах эквивалент хэлбэрээр илэрхийлж болно;

$A_1, \dots, A_m \leftarrow B_1, \dots, B_k$.

Програмын клаусыг мөн *бодит клаус* гэж нэрлэдэг бөгөөд зөвхөн нэг үгүйсгэгдээгүй атомтай өгүүлбэр. (12.1)-д үзүүлсэн хэлбэрээр бодит клаус нь үргэлж $m = 1$ байх бөгөөд энэ нь тодорхойлолт 12.3-т шууд оруулсан ойлголт юм. Тиймээс факт нь үгүйсгэгдсэн атом агуулаагүй тодорхой заалт юм. Эцэст нь, *query* эсвэл *goal* гэж нэрлэгддэг *эсрэг клаус* нь $m = 0$ байх хэлбэрийн (12.1) клаус юм.

12.3 Theory of Unification

Логик програмчлалын үндсэн тооцооллын механизм нь нөхцөл хоорондын тэгшитгэлийг нэгтгэх (unification) процедурыг ашиглан шийдвэрлэх явдал юм. Энэ процедурт хувьсагч болон нөхцлийг холбох (эсвэл үүсгэгдэх) боломжтой байхаар орлуулгыг тооцдог. Тооцооллын явцад олж авсан янз бүрийн орлуулгын найруулга нь тооцооллын үр дүнг өгдөг. Логик програмчлалын тооцооллын загварыг нарийвчлан үзэхийн өмнө бид нэгтгэх талаар бага зэрэг нарийвчлан шинжлэх ёстой, энэ хэсэгт бидний хийх ажил.

12.3.1 The Logic Variable

Бидний авч үзэж буй хувьсагчийн тухай ойлголт нь 6.2.1-д дурдсанаас өөр гэдгийг эхлээд тодруулъя. Энд бид урьдчилан тогтоосон олонлогоос утгыг авч болох үл мэдэгдэх *логик хувьсагчийг* авч үзье. Манай тохиолдолд энэ олонлог нь өгөгдсөн цагаан толгойн дээрх тодорхой нөхцөл. Энэ баримт нь логик програмчлалын логик хувьсагчийн хэрэглээтэй хамтдаа логик хувьсагч болон императив хэлний засварлагдах боломжтой хувьсагч хооронд гурван чухал ялгааг бий болгодог.

1. Логик хувьсагчийг зөвхөн нэг удаа холбож болно,, өөрөөр хэлбэл хувьсагч нь нөхцөлтэй холбогдсон бол энэ холболтыг устгах боломжгүй (гэхдээ энэ нөхцлийг доор тайлбарласнаар өөрчилж болно). Жишээлбэл, хэрэв бид X хувьсагчийг a тогтмолтой логик программд холбох юм бол дараа нь X -ийг b тогтмолтой холбодог өөр холбоосоор сольж болохгүй. Императив хэл дээр утга оноолтыг ашиглан энэ нь боломжтой нь ойлгомжтой. Хувьсагчийн холболтыг арилгах боломжгүй гэдэг нь хувьсагчийн утгыг өөрчлөх боломжгүй гэсэн үг биш юм. Энэхүү илэрхий зөрчилдөөнийг дараагийн хэсэгт авч үзэх болно.
2. Логик хувьсагчийн утгыг хэсэгчлэн тодорхойлж болно (эсвэл тодорхойгүй байж болно), дараа нь зааж өгөх боломжтой. Учир нь хувьсагчтай холбогдсон нөхцөл нь эргээд бусад логик хувьсагчдыг агуулж болно. Жишээлбэл, X хувьсагч нь $f(Y, Z)$ нөхцөлд холбогдсон бол Y ба Z хувьсагчдын залгамжлах холболт нь X хувьсагчийн утгыг мөн өөрчилнө: хэрэв Y нь a -тай, Z нь $g(W)$ -тэй холбогдсон бол X -ийн утга нь $f(a, g(W))$ болно. W -ийн утгыг өөрчлөх замаар энэ явцыг үргэлжлүүлж болно. Хувьсагчийн утгыг залгамжлан ойртуулах замаар тодорхойлох энэхүү

механизм нь логик хэлний хувьд энгийн зүйл бөгөөд утга оноогдсон утгыг хэсэгчлэн тодорхойлох боломжгүй императив хэлэнд тохиолддог харгалзах механизмаас арай өөр юм.

3. Гурав дахь чухал ялгаа нь логик хувьсагчдын холболтын хоёр чиглэлтэй шинж чанартай холбоотой юм. Хэрэв X нь $f(Y)$ нөхцөлтэй холбогдож, дараа нь бид X -г $f(a)$ нөхцөлд холбож чадвал Y хувьсагчийг а тогтмолтой холбох үр нөлөө гарна. Энэ нь эхний зүйлтэй зөрчилдөхгүй, учир нь X -ийн $f(Y)$ нөхцлийн холболтыг устгаагүй, харин $f(Y)$ утгыг Y -ийн холболтоор зааж өгсөн болно. Тиймээс бид хувьсагчийн холбогдсон нөхцлийг өөрчилснөөр түүний утгыг өөрчлөхөөс гадна тухайн хувьсагчийн өөр нэг холбоосыг өгснөөр энэ нөхцлийг өөрчилж болно. Энэ хоёр дахь холболт нь эхнийхтэй тохирч байх ёстой, өөрөөр хэлбэл, хэрэв X нь $f(Y)$ гэсэн нэр томьёотой холбогдсон бол бид X -г $g(Z)$ хэлбэрийн гишүүнтэй холбохыг оролдож болохгүй нь ойлгомжтой.

Сүүлийнх нь суурь үндэс бөгөөд энэ нь бидэнд нэг логик програмыг огт өөр аргаар ашиглах боломжийг олгодог бөгөөд үүнийг доор харуулав. Үндсэндээ бид нэгтгэх механизмаас үүдэлтэй шинж чанарын тухай ярьж байгаа бөгөөд удахгүй хэлэлцэх болно. Юуны өмнө бид орлуулалтын тухай ойлголтыг нэвтрүүлэх ёстой.

12.3.2 Substitution

Хувьсагч ба нөхцөл хоорондын холбоо нь орлуулах ойлголтын өнцгөөс хийгдсэн байдаг бөгөөд нэрээс нь харахад хувьсагчийг нөхцлөөр "орлуулах" боломжийг олгодог. Ихэвчлэн θ , σ , ρ , ... гэсэн грек үсгээр тэмдэглэгдсэн орлуулгыг дараах байдлаар тодорхойлж болно.

Тодорхойлолт 12.4 (Орлуулга) Орлуулга гэдэг нь хувьсагчаас нөхцөл руу чиглэсэн функц бөгөөд өөрөөр хэлбэл, өөрсдөд нь харгалздаггүй хувьсагчийн тоо төгсгөлөг байдаг. Бид θ орлуулгыг дараах хэлбэрээр тэмдэглэнэ.

$$\theta = \{X_1/t_1, \dots, X_n/t_n\}$$

Энд $X_1, \dots, X_n$ нь өөр хувьсагч, $t_1, \dots, t_n$ нь нөхцөл бөгөөд $i = 1, \dots, n$ -ийн хувьд t_i нь X_i -ээс ялгаатай гэж үздэг.

Өмнөх тодорхойлолтод X_1/t_1 хосыг *холбогдсон* гэж хэлсэн. Бүх $t_1, \dots, t_n$ нь *ground* нөхцөл байх тохиолдолд θ нь *ground орлуулга* гэнэ. Бид хоосон орлуулгын хувьд ϵ гэж бичдэг. Тодорхойлолт 12.4-т үзүүлсэн θ -ийн хувьд бид орлуулгын домэйн, кодомэйн болон хувьсагчдыг дараах байдлаар тодорхойлно.

$$\text{Domain}(\theta) = \{X_1, \dots, X_n\},$$

$$\text{Codomain}(\theta) = \{Y \mid Y \text{ a variable in } t_i, \text{ for some } t_i, 1 \leq i \leq n\}.$$

Орлуулгыг нөхцөл, ерөнхийдөө аливаа синтакс илэрхийлэлд хэрэглэж, орлуулах домэйнд байгаа хувьсагчдын утгыг өөрчлөх боломжтой.

12.3.3 Most General Unifier

Логик парадигмын тооцооллын үндсэн механизм нь $s = t$ хэлбэрийн тэгшитгэлийн үнэлгээ бөгөөд s ба t нь нөхцөл, " $=$ " нь бүх *ground* нөхцлийн олонлог дээрх синтаксик тэнцүү байдал гэж

тайлбарлагддаг предикатын тэмдэг; Энэ олонлогийг Хербранд юниверс гэж нэрлэдэг. Бид энэ тэгнцүү байдлыг илүү тодорхой болгохыг хичээх болно.

12.3.4 A Unification Algorithm

12.4 The Computational Model

"Дедункц тооцоолол" гэсэн санааг хэрэгжүүлдэг логик парадигм нь бидний өнөөг хүртэл үзсэн бусад бүх загвараас эрс ялгаатай тооцооллын загварыг ашигладаг. Синтез хийхийг хүсвэл бид бусад парадигмуудаас дараахь үндсэн ялгааг тодорхойлж чадна.

1. Наад зах нь цэвэр загварт байж болох цорын ганц утга нь өгөгдсөн тэмдэглээн дээрх нөхцөл юм.
2. Программ нь бүхэлдээ логиктой декларатив уншилттай эсвэл үйлдлийн хэв маягийн процедур уншилттай байж болно.
3. Тооцоолол нь нөхцөл (тиймээс зорилгод) дээр гарч ирж буй хувьсагчдыг нэгтгэх механизмыг ашиглан бусад нөхцөл рүү оруулах замаар ажилладаг.
4. Хяналт нь бүхэлдээ автоматаар буцаах (backtracking) процесс дээр суурилсан хийсвэр машинаар (Prolog-д байдаг зарим боломжит тэмдэглэгээг эс тооцвол) удирдуулдаг.

12.4.1 The Herbrand Universe

12.4.2 Declarative and Procedural Interpretation

12.4.3 Procedure Calls

12.4.4 Control: Non-determinism

12.4.5 Some Examples

12.5 Extensions

12.5.1 Prolog

12.5.2 Logic Programming and Databases

12.6 Advantages and Disadvantages of the Logic Paradigm